

Testeo TechMind

Federico G. Gutierrez
Tester QA

VERSIÓN 3.0

CONTENIDOS

- 1** Informe de resultados
- 2** Detalle de casos
- 3** Próximas tareas

Informe de resultados

Proyecto: TechMind - Organización Inteligente del Conocimiento Técnico

Responsable QA: Federico G. Gutierrez
Fecha de Ejecución: 29 de Julio de 2026

Resumen Ejecutivo

Durante los Sprints 1, 2 y 3, se ejecutaron de manera exhaustiva las suites de pruebas funcionales, de integridad, seguridad y resiliencia sobre el stack completo, abarcando las APIs backend (FastAPI y Spring Boot), la base de datos PostgreSQL (techmind) y la integración con el microservicio de Machine Learning.

- **Sprint 1 (FastAPI & PostgreSQL):** Se ejecutó la suite completa de pruebas sobre los endpoints de la API FastAPI y su integración con PostgreSQL, logrando una cobertura total en escenarios funcionales, de límites, seguridad, resiliencia y validación de datos.
- **Sprint 2 (Data QA - PostgreSQL):** Se auditó la integridad y persistencia de la base de datos techmind, evaluando las tablas contenidos y predicciones. Se logró cobertura total en integridad referencial, consistencia de esquemas, unicidad de claves primarias, límites superiores e inferiores y resiliencia ante inyecciones de datos.
- **Sprint 3 (Backend Spring Boot & ML):** Se validó la capa de API backend en Spring Boot conectada a PostgreSQL y al microservicio de ML, evaluando el flujo punta a punta desde las solicitudes HTTP POST (http://localhost:8080) y la invocación del modelo, hasta la respuesta JSON y la persistencia en contenidos.

En todas las etapas se alcanzó un 100% de cobertura y éxito en los casos planificados, garantizando la estabilidad del sistema ante grandes volúmenes de texto, la sanitización de caracteres y el cumplimiento estricto de esquemas.

47 PLANIFICADOS	47 EJECUTADOS	47 PASÓ	0 FALLÓ	100% ÉXITO
--------------------	------------------	------------	------------	---------------

Desglose por Tipo de Prueba (FastAPI)

Categoría	Planificados	PASÓ	FALLÓ	% Éxito
Funcionales (Flujo Feliz)	8	8	--	100
Casos Borde / Encoding (UTF-8)	3	3	--	100
Validación de Esquema / Tipos	3	3	--	100
Seguridad (Inyección SQL / Content-Type)	2	2	--	100
Endpoints Complementarios	2	2	--	100
Integridad de Datos	6	6	--	100

Conclusión y Recomendaciones

El MVP cumple con todos los criterios de aceptación funcionales, de rendimiento (< 2000 ms) y de seguridad especificados para el Sprint 1. La API demuestra alta estabilidad y resiliencia ante errores de infraestructura.

Desglose por Tipo de Prueba (PostgreSQL - techmind)

Categoría	Planificados	PASÓ	FALLÓ	% Éxito
Compleitud	5	5	--	100
Integridad y Estructura	6	6	--	100
Formato y Calidad de Texto	3	3	--	100
Edge Cases (Límites y Vacíos)	2	2	--	100
Rendimiento e Inserción Masiva	1	1	--	100

Conclusión y Recomendaciones

La capa de datos en PostgreSQL demostró total solidez técnica en el Sprint 1 al garantizar la integridad de los datos (claves primarias, campos obligatorios y unicidad), alta resiliencia y seguridad (protección contra SQLi y soporte de estructuras complejas/TEXT) y excelente escalabilidad frente a inserciones masivas y grandes volúmenes de datos.

Desglose por Tipo de Prueba (Spring Boot)

Categoría	Planificados	PASÓ	FALLÓ	% Éxito
Clasificación y Persistencia Core	1	1	--	100
Codificación y Caracteres (UTF-8)	1	1	--	100
Validación de Campos y Esquema	2	2	--	100
Seguridad y Sanitización	1	1	--	100
Edge Cases (Textos de Gran Longitud)	1	1	--	100

Conclusión y Recomendaciones

El módulo Spring Boot API demostró una alta madurez técnica e integración funcional al responder de manera óptima en seguridad, rendimiento e integridad. La aplicación intercepta entradas inválidas en la capa del servidor mediante Bean Validation retornando errores 400 Bad Request, previene ataques de inyección SQL mediante consultas parametrizadas con JPA/Hibernate, preserva símbolos y emojis gracias a su configuración UTF-8, y procesa de forma transparente payloads superiores a los 50.000 caracteres dentro de los tiempos de respuesta esperados.

Detalle de casos

Sprint 1 (FastAPI & PostgreSQL)

Testeo Funcionales (Flujo Feliz) — [8 casos]

- **CP-FASTAPI-01:** Valida que la API clasifique correctamente un contenido técnico válido devuelto con HTTP 200.
- **CP-FASTAPI-02:** Comprueba la extracción y retorno correcto de la lista de palabras clave relevantes en la respuesta.
- **CP-FASTAPI-03:** Verifica la generación de un score de probabilidad de clasificación válido entre 0 y 1.
- **CP-FASTAPI-09:** Confirma que la estructura del objeto JSON devuelto contenga todas las claves y tipos de datos especificados.
- **CP-FASTAPI-10:** Valida la correcta inicialización y carga de los modelos serializados (.joblib) al arrancar el servicio.
- **CP-FASTAPI-11:** Mide el tiempo de respuesta del endpoint asegurando que la inferencia se ejecute en menos de 2000 ms.
- **CP-FASTAPI-20:** Verifica que el modelo normalice y procese adecuadamente textos ingresados con tildes y mayúsculas sostenidas.
- **CP-FASTAPI-22:** Evalúa la latencia, estabilidad y cero tasa de errores del endpoint ante 100 peticiones concurrentes en ráfaga.

Casos Borde / Encoding (UTF-8) — [3 casos]

- **CP-FASTAPI-12:** Evalúa el comportamiento y la gestión de memoria de la API ante payloads de gran tamaño (+500k caracteres).
- **CP-FASTAPI-13:** Garantiza la sanitización y el soporte de codificación UTF-8 procesando caracteres especiales, etiquetas y emojis sin errores.
- **CP-FASTAPI-21:** Evalúa cómo reacciona la API ante entradas no técnicas o sin sentido.

Validación de Esquema / Tipos — [3 caso]

- **CP-FASTAPI-14:** Confirma el rechazo inmediato (HTTP 422) por parte de Pydantic al enviar tipos de datos no válidos (números o booleanos).
- **CP-FASTAPI-23:** Valida que la API rechace payloads nulos devolviendo 422 Unprocessable Entity cuando el cuerpo enviado está vacío.
- **CP-FASTAPI-24:** Valida que la API detecte errores de sintaxis JSON y rechace la petición con un 422 Unprocessable Entity.

Seguridad (Inyección SQL / Content-Type) — [2 casos]

- **CP-FASTAPI-15:** Comprueba la inmunidad ante intentos de inyección SQL guardando los comandos maliciosos como texto plano.
- **CP-FASTAPI-16:** Valida el rechazo de solicitudes con formatos no soportados como XML mediante una respuesta HTTP 422.

Endpoints Complementarios — [2 casos]

- **CP-FASTAPI-17:** Verifica la disponibilidad y estado operativo del microservicio mediante el endpoint de diagnóstico GET /health.
- **CP-FASTAPI-18:** Valida que el endpoint GET /categorias retorne el catálogo completo con las 8 categorías del modelo.

Integridad de Datos — [6 casos]

- **CP-FASTAPI-04:** Controla el rechazo de la solicitud (HTTP 422) cuando el campo "titulo" se envía vacío.
- **CP-FASTAPI-05:** Controla el rechazo de la solicitud (HTTP 422) cuando el campo "texto" no contiene información.
- **CP-FASTAPI-06:** Valida la respuesta de error adecuada al enviar tanto el título como el texto vacíos en el payload.
- **CP-FASTAPI-07:** Comprueba la interceptación y manejo de excepciones ante la recepción de una estructura JSON mal formada.
- **CP-FASTAPI-08:** Verifica el rechazo con código HTTP 405 Method Not Allowed al enviar una petición GET al endpoint POST /predecir.
- **CP-FASTAPI-19:** Verifica que la API rechace la petición cuando se omiten claves obligatorias dentro del cuerpo JSON.

Sprint 2 (Data QA - PostgreSQL)

Complejidad — [5 casos]

- **CP-DB-01:** Verifica la ausencia de filas incompletas en contenidos.
- **CP-DB-02:** Confirma de que no existen registros de texto idénticos sobrecargando la base de datos.
- **CP-DB-03:** Distribución uniforme y validación sobre las 8 categorías temáticas del modelo.
- **CP-DB-04:** Coincidencia del 100% entre la respuesta JSON de la API y el registro persistido en predicciones.
- **CP-DB-05:** Aprobación de rango numérico estrictamente acotado en [0,0; 1,0].

Integridad y Estructura — [6 casos]

- **CP-DB-08:** 100% de unicidad en la columna id de predicciones con secuencia autoincremental coherente.
- **CP-DB-09:** Fechas de la columna created_at válidas, no nulas y alineadas a la zona horaria actual.
- **CP-DB-10:** Confirmación de cadenas válidas y limpias sin presencia de nulos ni espacios aislados.
- **CP-DB-11:** Categorías predichas pertenecientes de forma estricta al dominio semántico (Backend, Bases de Datos, etc.).
- **CP-DB-13:** Coincidencia perfecta del 100% en campos clave de la tabla de origen contenidos.
- **CP-DB-14:** Verifica las restricciones PRIMARY KEY sobre la tabla contenidos.

Formato y Calidad de Texto — [3 caso]

- **CP-DB-06:** Valida el tipo de dato text[] para palabras clave e inferencias con arreglos vacíos {}.
- **CP-DB-07:** Sanitización e inserción exitosa del texto literal conteniendo comandos destructivos (ej. '; DROP TABLE...') sin alteración de la estructura.
- **CP-DB-15:** Evaluación cualitativa de textos cortos garantizando significancia técnica para el modelo NLP.

Edge Cases (Límites y Vacíos) — [2 casos]

- **CP-DB-16:** Validación de rechazo/ausencia de textos con longitud 0 o TRIM nulo.
- **CP-DB-17:** Confirmación de persistencia íntegra de payloads extensos en el tipo de dato TEXT sin truncamiento.

Rendimiento e inserción masiva — [1 caso]

- **CP-DB-12:** Resistencia y persistencia exitosa ante ráfagas de carga masiva de hasta 300 peticiones/minuto.

Sprint 3 (Backend Spring Boot & ML)

Clasificación y Persistencia Core — [1 casos]

- **CP-SPRINGBOOT-01:** Valida que la API clasifique e invoque el modelo de ML, responda con HTTP 201 y persista el contenido en PostgreSQL.

Codificación y Caracteres (UTF-8) — [1 caso]

- **CP-SPRINGBOOT-02:** Comprueba la preservación correcta de codificación UTF-8, tildes y emojis en la respuesta JSON y en la base de datos.

Validación de Campos y Esquema — [2 casos]

- **CP-SPRINGBOOT-03:** Verifica el rechazo de peticiones sin el campo obligatorio título, devolviendo un error HTTP 400 Bad Request.
- **CP-SPRINGBOOT-04:** Confirma que el backend invalide y rechace mediante @NotBlank las entradas con cadenas vacías o solo espacios.

Seguridad y Sanitización — [1 caso]

- **CP-SPRINGBOOT-05:** Evalúa la sanitización del ORM ante comillas simples e inyecciones SQL, garantizando la integridad de la base de datos.

Edge Cases (Textos de Gran Longitud) — [1 caso]

- **CP-SPRINGBOOT-06:** Mide el rendimiento y la persistencia sin truncamiento al procesar payloads extensos de más de 50,000 caracteres.

Matriz

Evidencias

Próximas tareas

Para continuar fortaleciendo la calidad y madurez del proyecto, se proponen las siguientes iniciativas para la próxima etapa:

- **Automatización en Pipeline CI/CD (GitHub Actions + Newman):** Integrar la ejecución automática de la suite de pruebas de Postman en cada despliegue o pull request, asegurando la detección temprana de regresiones antes de pasar a entornos de producción.
- **Monitoreo Continuo y Alertas en Producción (OCI):** Configurar un esquema de lectura centralizada de logs en Oracle Cloud Infrastructure para alertar de forma automática ante caídas de servicio, errores 500 o fallos de conexión a la base de datos.